

小结

168

C++语言提供了一套丰富的运算符，并定义了这些运算符作用于内置类型的运算对象时所执行的操作。此外，C++语言还支持运算符重载的机制，允许我们自己定义运算符作用于类类型时的含义。第14章将介绍如何定义作用于用户类型的运算符。

对于含有超过一个运算符的表达式，要想理解其含义关键要理解优先级、结合律和求值顺序。每个运算符都有其对应的优先级和结合律，优先级规定了复合表达式中运算符组合的方式，结合律则说明当运算符的优先级一样时应该如何组合。

大多数运算符并不明确规定运算对象的求值顺序：编译器有权自由选择先对左侧运算对象求值还是先对右侧运算对象求值。一般来说，运算对象的求值顺序对表达式的最终结果没有影响。但是，如果两个运算对象指向同一个对象而且其中一个改变了对象的值，就会导致程序出现不易发现的严重缺陷。

最后一点，运算对象经常从原始类型自动转换成某种关联的类型。例如，表达式中的小整型会自动提升成大整型。不论内置类型还是类类型都涉及类型转换的问题。如果需要，我们还可以显式地进行强制类型转换。

术语表

算术转换 (arithmetic conversion) 从一种算术类型转换成另一种算术类型。在二元运算符的上下文中，为了保留精度，算术转换通常把较小的类型转换成较大的类型（例如整型转换成浮点型）。

结合律 (associativity) 规定具有相同优先级的运算符如何组合在一起。结合律分为左结合律（运算符从左向右组合）和右结合律（运算符从右向左组合）。

二元运算符 (binary operator) 有两个运算对象参与运算的运算符。

强制类型转换 (cast) 一种显式的类型转换。

复合表达式 (compound expression) 含有多于一个运算符的表达式。

const_cast 一种涉及 const 的强制类型转换。将底层 const 对象转换成对应的非常量类型，或者执行相反的转换。

转换 (conversion) 一种类型的值改变成另一种类型的值的过程。C++语言定义了内置类型的转换规则。类类型同样可以转换。

dynamic_cast 和继承及运行时类型识别一

起使用。参见 19.2 节（第 730 页）。

表达式 (expression) C++程序中最低级别的计算。表达式将运算符作用于一个或多个运算对象，每个表达式都有对应的求值结果。表达式本身也可以作为运算对象，这时就得到了对多个运算符求值的复合表达式。

隐式转换 (implicit conversion) 由编译器自动执行的类型转换。假如表达式需要某种特定的类型而运算对象是另外一种类型，此时只要规则允许，编译器就会自动地将运算对象转换成所需的类型。

整型提升 (integral promotion) 把一种较小的整数类型转换成与之最接近的较大整数类型的过程。不论是否真的需要，小整数类型（即 short、char 等）总是会得到提升。

左值 (lvalue) 是指那些求值结果为对象或函数的表达式。一个表示对象的非常量左值可以作为赋值运算符的左侧运算对象。

运算对象 (operand) 表达式在某些值上执行运算，这些值就是运算对象。一个运算

169

符有一个或多个相关的运算对象。

运算符 (operator) 决定表达式所做操作的符号。C++语言定义了一套运算符并说明了这些运算符作用于内置类型时的含义。C++还定义了运算符的优先级和结合律以及每种运算符处理的运算对象数量。可以重载运算符使其能处理类类型。

求值顺序 (order of evaluation) 是某个运算符的运算对象的求值顺序。大多数情况下, 编译器可以任意选择运算对象求值的顺序。不过运算对象一定要在运算符之前得到求值结果。只有 `&&`、`||`、条件和逗号四种运算符明确规定了求值顺序。

重载运算符 (overloaded operator) 针对某种运算符重新定义的适用于类类型的版本。第14章将介绍重载运算符的方法。

优先级 (precedence) 规定了复合表达式中不同运算符的执行顺序。与低优先级的运算符相比, 高优先级的运算符组合得更紧密。

提升 (promoted) 参见整型提升。

reinterpret_cast 把运算对象的内容解释成另外一种类型。这种强制类型转换本质上依赖于机器而且非常危险。

结果 (result) 计算表达式得到的值或对象。

右值 (rvalue) 是指一种表达式, 其结果是值而非值所在的位置。

短路求值 (short-circuit evaluation) 是一个专有名词, 描述逻辑与运算符和逻辑或运算符的执行过程。如果根据运算符的第一个运算对象就能确定整个表达式的结果, 求值终止, 此时第二个运算对象将不会被求值。

sizeof 是一个运算符, 返回存储对象所需的字节数, 该对象的类型可能是某个给定的类型名字, 也可能由表达式的返回结果确定。

static_cast 显式地执行某种定义明确的类型转换, 常用于替代由编译器隐式执行的类型转换。

一元运算符 (unary operators) 只有一个运算对象参与运算的运算符。

运算符 (, operator) 逗号运算符, 是一种从左向右求值的二元运算符。逗号运算符的结果是右侧运算对象的值, 当且仅当右侧运算对象是左值时逗号运算符的结果是左值。

? :运算符 (? : operator) 条件运算符, 以下述形式提供 if-then-else 逻辑的表达式

cond ? expr1 : expr2;

如果条件 *cond* 为真, 对 *expr1* 求值; 否则对 *expr2* 求值。*expr1* 和 *expr2* 的类型应该相同或者能转换成同一种类型。*expr1* 和 *expr2* 中只有一个会被求值。

&&运算符 (&& operator) 逻辑与运算符, 如果两个运算对象都是真, 结果才为真。只有当左侧运算对象为真时才会检查右侧运算对象。

&运算符 (& operator) 位与运算符, 由两个运算对象生成一个新的整型值。如果两个运算对象对应的位都是1, 所得结果中该位为1; 否则所得结果中该位为0。

^运算符 (^ operator) 位异或运算符, 由两个运算对象生成一个新的整型值。如果两个运算对象对应的位有且只有一个是1, 所得结果中该位为1; 否则所得结果中该位为0。

||运算符 (|| operator) 逻辑或运算符, 任何一个运算对象是真, 结果就为真。只有当左侧运算对象为假时才会检查右侧运算对象。

|运算符 (| operator) 位或运算符, 由两个运算对象生成一个新的整型值。如果两个运算对象对应的位至少有一个是1, 所得结果中该位为1; 否则所得结果中该位为0。

++运算符 (++ operator) 递增运算符。包括两种形式: 前置版本和后置版本。前置递增运算符得到一个左值, 它给运算符加1并得到运算对象改变后的值。后置递增运算符得到一个右值, 它给运算符加1并得到运算对象原始的、未改变的值的副本。注意: 即使迭代器没有定义++运算符, 也会

有++运算符。

--运算符 (-- operator) 递减运算符。包括两种形式：前置版本和后置版本。前置递减运算符得到一个左值，它从运算符减 1 并得到运算对象改变后的值。后置递减运算符得到一个右值，它从运算符减 1 并得到运算对象原始的、未改变的值的副本。注意：即使迭代器没有定义--运算符，也会有--运算符。

<<运算符 (<< operator) 左移运算符，将左侧运算对象的值的（可能是提升后的）副本向左移位，移动的位数由右侧运算对象确定。右侧运算对象必须大于等于 0 而且小于结果的位数。左侧运算对象应该是无符号类型，如果它是带符号类型，则一旦移动改变了符号位的值就会产生未定义的结果。

>>运算符 (>> operator) 右移运算符，除了移动方向相反，其他性质都和左移运算符类似。如果左侧运算对象是带符号类型，那么根据实现的不同新移入的内容也不同，新移入的位可能都是 0，也可能都是符号位的副本。

~运算符 (~ operator) 位求反运算符，生成一个新的整型值。该值的每一位恰好与（可能是提升后的）运算对象的对应位相反。

!运算符 (! operator) 逻辑非运算符，将它的运算对象的布尔值取反。如果运算对象是假，则结果为真，如果运算对象是真，则结果为假。

第 5 章

语句

内容

5.1 简单语句.....	154
5.2 语句作用域.....	155
5.3 条件语句.....	156
5.4 迭代语句.....	165
5.5 跳转语句.....	170
5.6 try 语句块和异常处理	172
小结	178
术语表.....	178

和大多数语言一样，C++提供了条件执行语句、重复执行相同代码的循环语句和用于中断当前控制流的跳转语句。本章将详细介绍 C++语言所支持的这些语句。